

Greedy Techniques, DP on Trees, DP with Bitmasking, and DP with Segments

Below are detailed study notes in a simple format suitable for beginners. Each topic contains:

- Definition
 - Key Idea
 - Step-by-step algorithm
 - Example 1 (Simple)
 - Example 2 (Real-world)
 - Time Complexity
 - Easy Python program
-

1. Greedy Techniques

What is a Greedy Algorithm?

A **Greedy Algorithm** solves a problem by making the **best choice at the current step** without thinking about future consequences.

"Choose the best option now and hope it leads to the overall best solution."

Unlike Dynamic Programming, Greedy never revisits previous decisions.

When can Greedy be Used?

Greedy works only when the problem satisfies:

1. Greedy Choice Property

Making the locally optimal choice always leads to the globally optimal solution.

2. Optimal Substructure

The optimal solution can be built from optimal solutions of smaller problems.

Steps

1. Sort the data (if needed).
 2. Choose the best available option.
 3. Never change the decision.
 4. Continue until finished.
-

Example 1: Coin Change Problem

Coins Available

₹10 ₹5 ₹2 ₹1

Amount = ₹28

Step 1

Take the largest coin.

28 → 10

Remaining = 18

Step 2

Take another ₹10.

Remaining = 8

Step 3

Take ₹5.

Remaining = 3

Step 4

Take ₹2.

Remaining = 1

Step 5

Take ₹1.

Coins used:

10 10 5 2 1

Only **5 coins** are used.

Python Program

```
coins = [10, 5, 2, 1]
amount = 28

result = []

for coin in coins:
    while amount >= coin:
        amount -= coin
        result.append(coin)

print(result)
```

Output

[10, 10, 5, 2, 1]

Example 2 (Real World): Activity Selection

A teacher has only one classroom.

Many students request presentation slots.

Activity	Start	End
A	1	3
B	2	4

Activity	Start	End
C	3	5
D	5	7

Choose the maximum number of **non-overlapping activities**.

Step 1

Sort activities by ending time.

- A
- B
- C
- D

Step 2

Choose A.

1 - 3

Step 3

B overlaps with A.

Skip B.

Step 4

Choose C.

3 - 5

Step 5

Choose D.

5 - 7

Final Answer

Python Program

```
activities = [(1, 3), (2, 4), (3, 5), (5, 7)]

activities.sort(key=lambda x: x[1])

selected = []
last_end = 0

for start, end in activities:
    if start >= last_end:
        selected.append((start, end))
        last_end = end

print(selected)
```

Output

```
[(1, 3), (3, 5), (5, 7)]
```

Time Complexity

Sorting

$O(n \log n)$

Selection

$O(n)$

Overall

$O(n \log n)$

2. Dynamic Programming on Trees

What is Tree DP?

Dynamic Programming is performed on tree nodes.

Each node stores information about its subtree.

Instead of recalculating, every subtree answer is computed only once.

Common Problems

- Maximum Path Sum
 - House Robber III
 - Diameter of Tree
 - Maximum Independent Set
-

General Idea

For every node,

`Answer (node)`

depends on

`Answer(left child)`

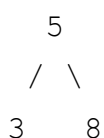
`Answer(right child)`

Process children first.

This is called **Postorder Traversal**.

Example 1: Sum of Tree

Tree



Goal:

Find the subtree sum.

Step 1

Visit left child.

3

Sum = 3

Step 2

Visit right child.

8

Sum = 8

Step 3

Visit root.

5 + 3 + 8

Answer:

16

Python Program

```
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def subtree_sum(root):
    if root is None:
        return 0

    left = subtree_sum(root.left)
```

```
        right = subtree_sum(root.right)

    return root.val + left + right

root = Node(5)
root.left = Node(3)
root.right = Node(8)

print(subtree_sum(root))
```

Output

16

Example 2 (Real World): House Robber III

A thief robs houses arranged in a tree.

He **cannot rob both a parent and its child**.

Tree

```
    5
   / \
  2   8
```

Option 1

Rob the root.

5

Cannot rob:

2

8

Total = **5**

Option 2

Skip the root.

Rob:

2 + 8

Total = **10**

Best Answer:

10

DP State

For every node:

- Rob this node
- Skip this node

Choose the maximum value.

Time Complexity

$O(n)$

3. DP with Bitmasking

What is Bitmasking?

A bitmask stores selected items using binary digits.

Example:

4 cities

0 1 2 3

Mask:

1011

Means:

- City 0 visited
 - City 1 visited
 - City 3 visited
 - City 2 not visited
-

Why Bitmask?

Instead of storing:

Visited List

Store:

One Integer

This is much faster and uses less memory.

Example 1: Traveling Salesman Problem

Cities:

A B C

Possible Masks:

001

011

111

Each mask represents visited cities.

DP State:

`dp[mask][city]`

Meaning:

Minimum cost to reach a city after visiting the cities represented by the mask.

Python Program

```
n = 3
```

```
for mask in range(1 << n):  
    print(bin(mask))
```

Output

```
0b0  
0b1  
0b10  
0b11  
0b100  
0b101  
0b110  
0b111
```

Example 2 (Real World): Delivery Robot

A robot delivers parcels to:

- House 1
- House 2
- House 3

Masks:

001

Only House 1 delivered.

011

House 1 and House 2 delivered.

111

All houses delivered.

The robot remembers delivered houses using one integer.

Time Complexity

Usually

$O(2^n \times n)$

4. DP with Segments (Interval DP)

What is Segment DP?

Dynamic Programming on intervals.

State:

`dp[left][right]`

Stores the answer for a segment.

Applications

- Matrix Chain Multiplication
 - Burst Balloons
 - Palindrome Partitioning
 - Optimal Binary Search Tree
-

Example 1: Matrix Chain Multiplication

Matrices:

A B C

Need minimum multiplication cost.

DP State:

`dp[i][j]`

Minimum cost from matrix **i** to **j**.

Step

Split every interval.

ABC

Possible splits:

A | BC

AB | C

Choose the minimum cost.

Python Program

```
arr = [10, 20, 30]

n = len(arr)

dp = [[0] * n for _ in range(n)]

print(dp)
```

Output

```
[[0, 0, 0],
 [0, 0, 0],
 [0, 0, 0]]
```

Example 2 (Real World): Cutting a Chocolate Bar

Chocolate:

ABCDE

Need the minimum cutting cost.

Possible cuts:

A | BCDE

AB | CDE

ABC | DE

DP computes every segment only once.

Finally, it chooses the minimum total cost.

General DP Formula

`dp[left][right]`

=

`minimum(`

`dp[left][k]`

+

`dp[k+1][right]`

+

`cost`

)

Time Complexity

Usually

$O(n^3)$

Comparison Table

Technique	Main Idea	DP State	Common Applications
Greedy	Choose the best option immediately	No DP table	Coin Change, Activity Selection, Huffman Coding
DP on Trees	Solve each subtree once	Node	House Robber III, Diameter, Tree Path Sum
DP with Bitmasking	Represent visited states using bits	<code>dp[mask][i]</code>	Traveling Salesman, Assignment Problem
DP with Segments	Solve intervals/subarrays	<code>dp[l][r]</code>	Matrix Chain Multiplication, Burst Balloons, Palindrome Partitioning

Short Summary

- **Greedy** → Makes the best local choice; decisions are never changed.
- **Tree DP** → Solve children first (Postorder Traversal), then compute the parent.
- **Bitmask DP** → Use binary bits to efficiently represent subsets or visited states.
- **Segment DP** → Break intervals into smaller intervals and combine their optimal solutions.